

Computer Security 2026

Side channel attacks

Roberto Guanciale
KTH

Often implementation weak link

— — —

- Bugs: buffer overflows

Often implementation weak link

— — —

- Bugs: buffer overflows
- Fault injection: active, partially invasive attack
 - Row-hammer
- Side channels: passive, non-invasive attacks

Often implementation weak link

— — —

- Bugs: buffer overflows
- Fault injection: active, partially invasive attack
 - Row-hammer
- Side channels: passive, non-invasive attacks

Crash course on information flow security

```
int secret;  
int public;
```

```
public = public+1;
```

Crash course on information flow security

```
int secret;  
int public;
```

```
public = secret+1;
```

Crash course on information flow security

```
int secret;      // high
int public;      // low
```

```
public = secret+1;
```

Crash course on information flow security

```
int secret;      // high
int public;      // low
```

```
public = public+1;
```

```
secret = public;
```

Crash course on information flow security

```
int secret;      // high
int public;      // low
```

```
if (secret > 0)
```

```
    public = 1;
```

Non interference

$s^1 \sim s^2$ iff they have same low (public) values

If $s^1 \sim s^2$ then $\text{program}(s^1) \sim \text{program}(s^2)$

Crash course on information flow security

```
int input;    // low
int authorised; // low
int pin;      // high
```

```
if (input == pin)
    authorised = true
```

If $s^1 \sim s^2$ then $\text{program}(s^1) \sim \text{program}(s^2)$

Crash course on information flow security

```
int input;    // low
int authorised; // low
int pin;      // high
```

```
if (input == pin)
```

```
    authorised = true
```

If $s^1 \sim s^2$ or $s^1 =^{\text{declass.}} s^2$ then $\text{program}(s^1) \sim \text{program}(s^2)$

$=^{\text{declass.}} (s^1.\text{input} == s^1.\text{pin}) == (s^2.\text{input} == s^2.\text{pin})$

Crash course on information flow security

```
int out;           // low
int [] salaries;  // high
```

```
out = 0
```

```
for s in salaries:
```

```
    out += s
```

Crash course on information flow security

```
int plaintext;    // high
int key;          // high
int ciphertext;   // low
```

```
ciphertext = AES(plaintext, key);
```

Crash course on information flow security

— — —

```
int plaintest;      // high
int key;            // high
int ciphertext;     // low

ciphertext = AES(plaintest, key);
```

IND-CPA

- Attacker chooses two low equivalent states
- We randomly ($p=\frac{1}{2}$) execute the program from one of the two states
- Attacker outputs which state has been used

For all (polynomial-time) attackers, the probability of winning the IND-CPA game is at most $\frac{1}{2} + \epsilon$, where ϵ is *negligible*.

Side channel formally

— — —

- Even if $s^1 \sim s^2$ and $\text{program}(s^1) \sim \text{program}(s^2)$, $\text{program}(s^1)$ distinguishable from $\text{program}(s^2)$

```
check-pin(pin):  
    for i in 0..3  
        if pin[i] != pwd[i]:  
            return False  
  
    return True
```

- 10K combinations
 - Avg 5K combinations
 - Probability guess in three attempts

```
check-pin(pin):  
    for i in 0..3  
        if pin[i] != pwd[i]:  
            return False  
  
    return True
```

- 10K combinations
 - Avg 5K combinations
 - Probability guess in three attempts $\sim 3/10000$

Timing side channel

```
check-pin(pin):  
    for i in 0..3  
        if pin[i] != pwd[i]:  
            return False  
    return True
```

- 10K combinations
 - Avg 5K combinations
 - Probability guess in three attempts $\sim 3/10000$
- Timing attack

```
T1=time(check-pin(pin1))  
pin2 = pin1[0]:=x  
T2=time(check-pin(pin2))  
if T1=T2 pin1[0] and pin[1] wrong  
if T1>T2 pin1[0] correct  
if T1<T2 pin2[0] correct
```

Timing side channel

```
check-pin(pin):  
    for i in 0..3  
        if pin[i] != pwd[i]:  
            return False  
  
    return True
```

- 10K combinations
 - Avg 5K combinations
 - Probability guess in three attempts $3/10000$
- Timing attack
- 10 WC per digit
 - avg 5
- Probability guess in three attempts $\sim 1/1000$

Modular exponentiation (i.e. RSA)

```
function modular_pow(base, exponent, modulus)
  if modulus = 1 then return 0
  result := 1
  base := base mod modulus
  while exponent > 0
    if (exponent mod 2 == 1):
      result := (result * base) mod modulus
    exponent := exponent >> 1
    base := (base * base) mod modulus
  return result
```

Modular exponentiation (i.e. RSA)

```
function modular_pow(base, exponent, modulus)
  if modulus = 1 then return 0
  result := 1
  base := base mod modulus
  while exponent > 0
    if (exponent mod 2 == 1):
      result := (result * base) mod modulus
    exponent := exponent >> 1
    base := (base * base) mod modulus
  return result
```

Non time based side channel

```
register(username):
```

```
    new-id = (select max(id) from users) + 1;
```

```
    insert into users (new-id, username);
```

```
    return new-id
```

Non time based side channel

```
bid(usr, value):
```

```
    if (value > max_value)
```

```
        insert into log (new-log-id1, "highest bid {value}")
```

```
    insert into log (new-log-id2, "{usr} bid is {value}")
```

```
    return new-log-id2
```

Side channel attacks

— — —

- Timing
 - Execution time of the victim
 - E.g. time of a visit with the doctor -> severity of health issue
 - Execution time of the attacker
 - E.g. time needed book an appointment -> number of patients
- Power consumption, Electromagnetic radiation
- Sound, temperature, accelerometer
 - “Practicality of accelerometer side channels on smartphones”
- ...

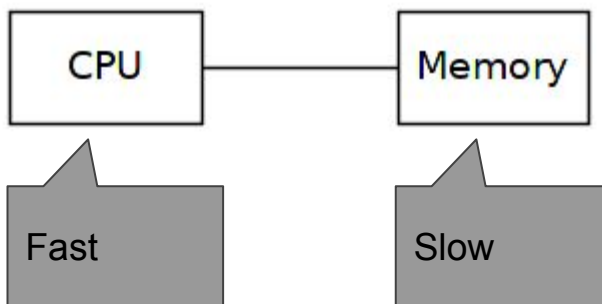
Side channel attacks

— — —

- Timing
 - Execution time of the victim
 - E.g. time of a visit with the doctor -> severity of health issue
 - Execution time of the attacker
 - E.g. time needed book an appointment -> number of patients
 - Caches
- Power consumption, Electromagnetic radiation
- Sound, temperature, accelerometer
- ...

CPU architecture

— — —

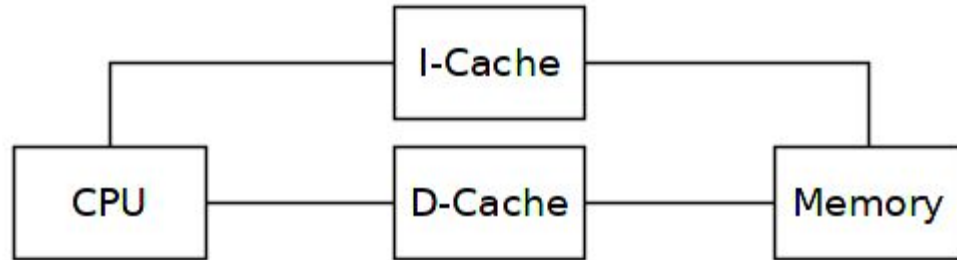


CPU architecture

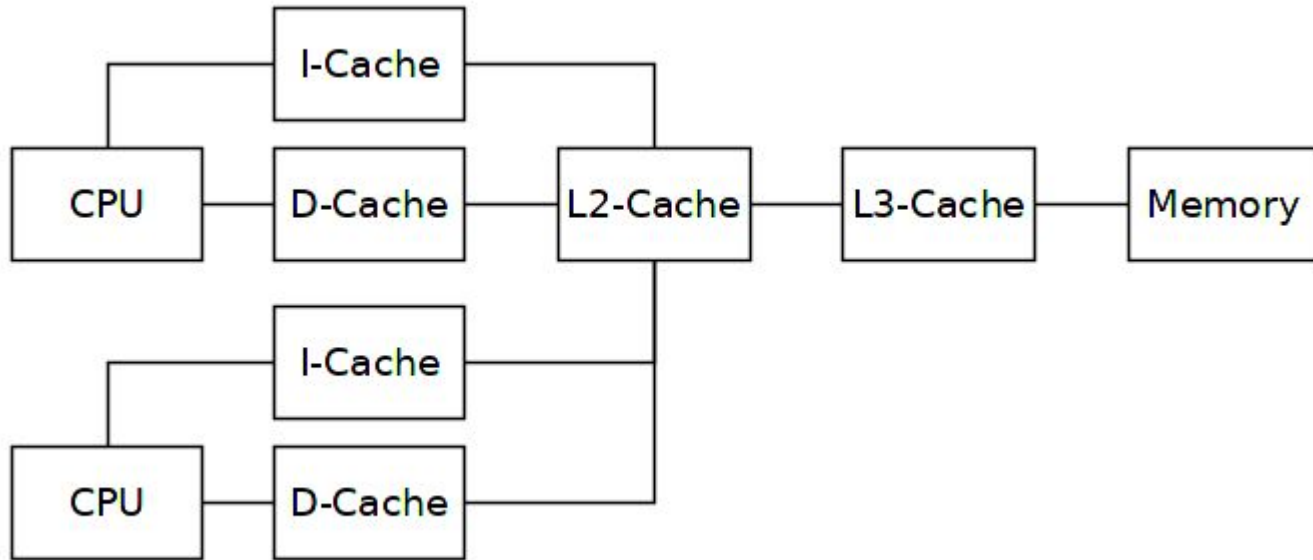
— — —



CPU architecture

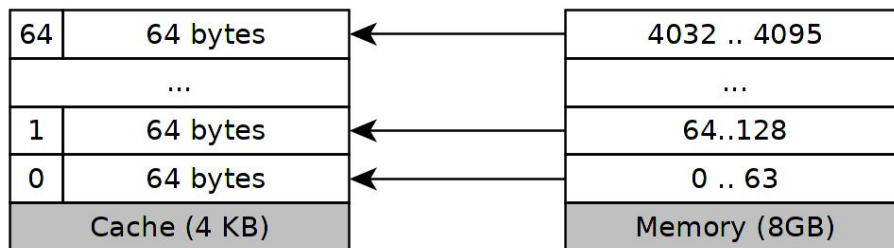


CPU architecture



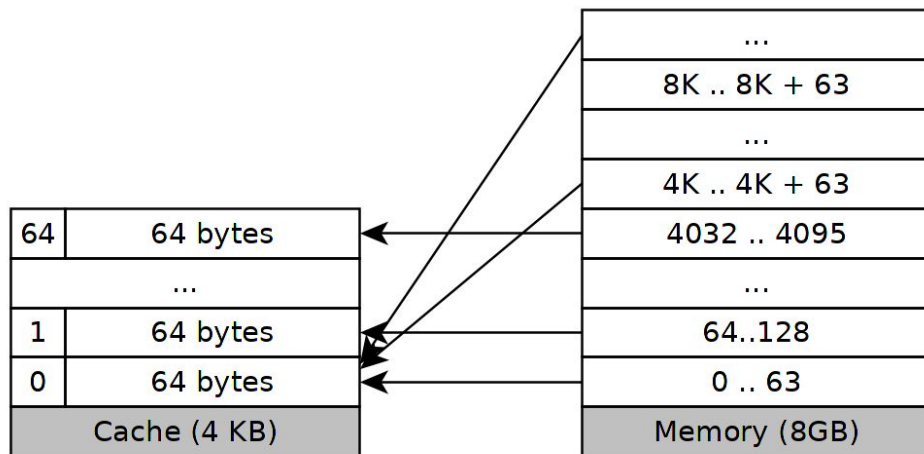
Cache

— — —



Cache

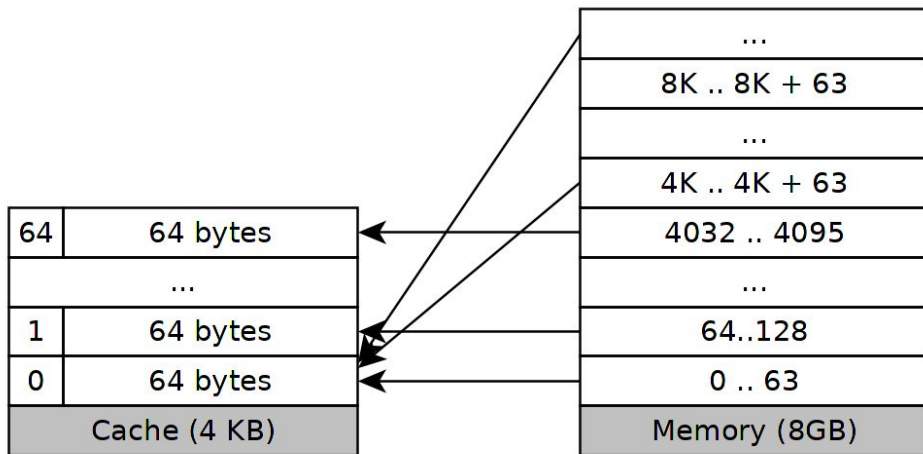
— — —



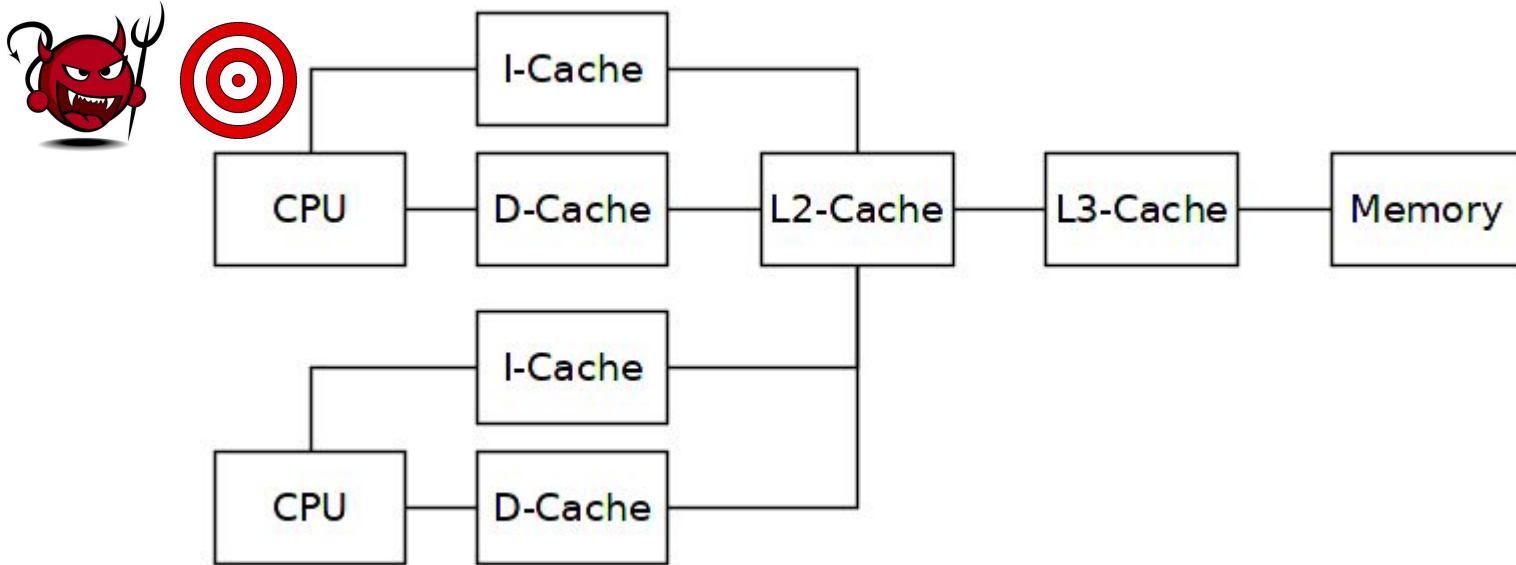
Cache

— — —

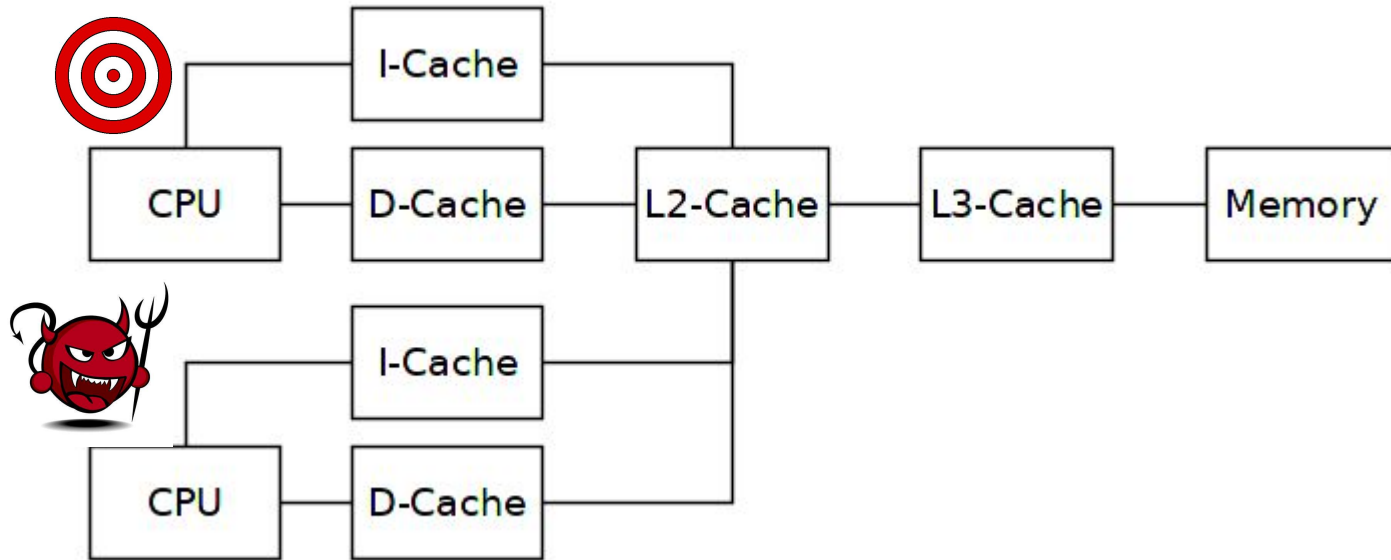
- When we access address 0, CPU
 - fetches 64 bytes into line 0
- When we load address 4KB (cache miss), CPU
 - evicts line 0
 - if dirty, writes back the content
- Multiway, VT/PT index, write through, ...
- Cache are transparent
- Cache are shared
- There are collisions
- Cache misses require more time



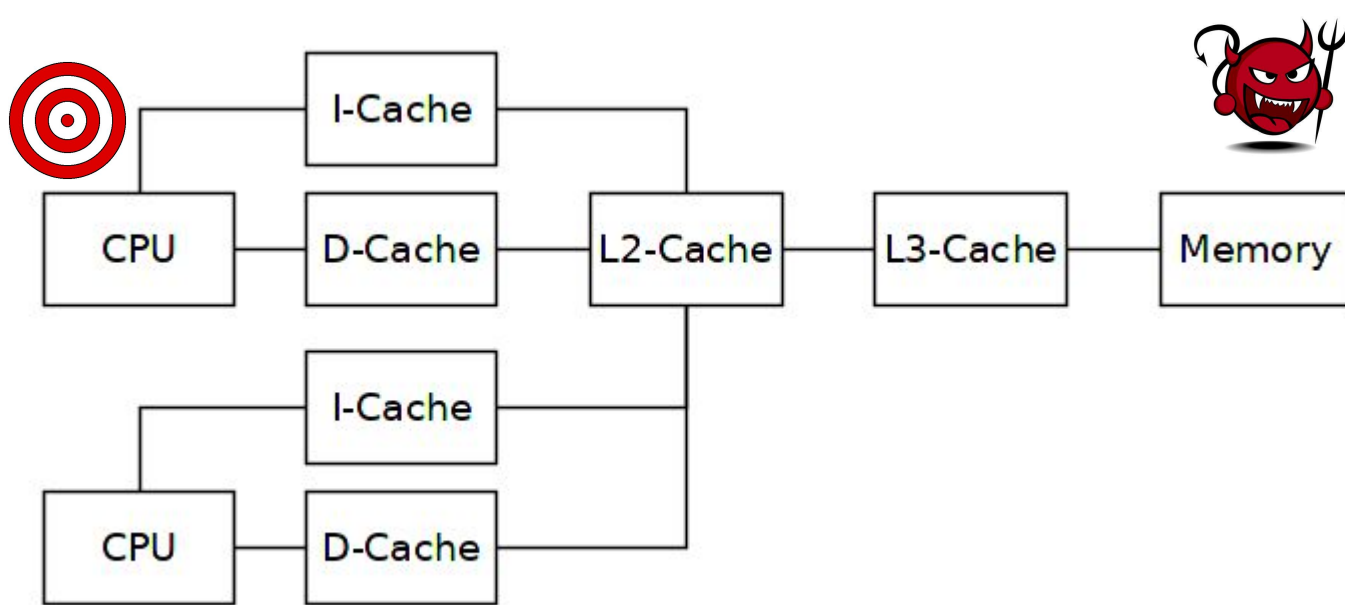
CPU architecture



CPU architecture



CPU architecture



Type of vulnerabilities: Time driven attacks

— — —

- Attacker triggers an execution of the victim (i.e. encryption)
- measures (indirectly or directly) the execution time
- uses this knowledge to estimate the number of cache misses and hits of the victim
- from number of hits/misses it estimate a secret value

```
victim():  
    v1 = M[4K]  
    v2 = M[4K+64*Secret]  
    save v1 + v2  
    return 0
```

If more than one cache misses then
Secret is not 0

Type of vulnerabilities: Access driven attacks

— — —

- Attacker determines the cache lines accessed by the victim
- By observing cache side effects of victim's computation on the behaviour of the attacker

```
victim():  
    v = M[4K + 64 * secret]  
    return 0
```

```
attacker-probe-shared-mem():  
    T0 = time()  
    x = M[4K]  
    T1 = time()-T0
```

Type of vulnerabilities: Access driven attacks

— — —

- Attacker determines the cache lines accessed by the victim
- By observing cache side effects of victim's computation on the behaviour of the attacker

```
attacker-prime():  
    x = M[8K]
```

```
victim():  
    v = M[4K + 64 * secret]  
    return 0
```

```
attacker-probe():  
    T0 = time()  
    x = M[8K]  
    T1 = time()-T0
```

Type of vulnerabilities: Access driven attacks

— — —

- Attacker determines the cache lines accessed by the victim
- By observing cache side effects of victim's computation on the behaviour of the attacker

```
victim():  
    v = M[4K + 64 * secret]  
    return 0
```

```
attacker-prime():  
    for l : 0 .. 63:  
        x = M[8K + 64*i]  
attacker-probe():  
    for l : 0 .. 63:  
        T0 = time()  
        x = M[8K + 64*i]  
        T1 = time()-T0
```

Type of vulnerabilities: Trace driven attacks

— — —

- Attacker has more capabilities
- Profile the cache activities during the execution of the victim
- thus observe the cache effects of a particular operation

```
victim():  
    v = M[4K]  
    if secret  
        v += M[8K]  
        v += M[8K+4]  
    else  
        v += M[4K+4]  
        v += M[8K]  
    return 0
```

```
If secret Miss-Miss-Hit  
otherwise Miss-Hit-Miss
```

Type of vulnerabilities: Instruction cache

— — —

- Attacker determines the cache lines accessed by the victim
- By observing cache side effects of victim's computation on the behaviour of the attacker

```
victim():  
    L:  If secret:  
        Jmp L1  
        return 0  
L1: return 0
```

Practical problems for attacker

— — —

- Noise
 - Other processes may interfere
- Multiple accesses make difficult to extract secrets from eviction patterns
- Is this a threat for real SW?

Access Driven Attack to AES

Advances on Access-Driven
Cache Attacks on AES

2006

— — —

Advanced Encryption Standard

— — —

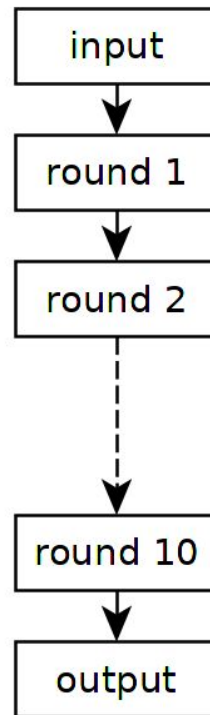
- Symmetric cypher
- Brute force 128-bit key
 - 1 decryption/ μ s = 264 billion years
- Each 128 bits block seen as 4x4 bytes matrix

B[0]	B[4]	B[8]	B[12]
B[1]	B[5]	B[9]	B[13]
B[2]	B[6]	B[10]	B[14]
B[3]	B[7]	B[11]	B[15]

Advanced Encryption Standard

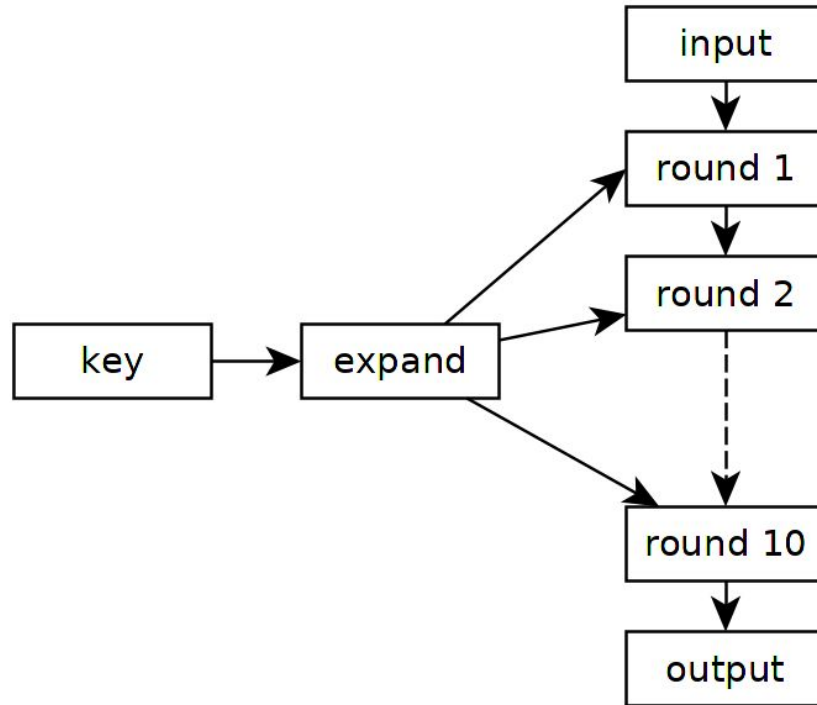
— — —

- Symmetric cypher
- Brute force 128-bit key
 - 1 decryption/ μ s = 264 billion years
- Each 128 bits block seen as 4x4 bytes matrix



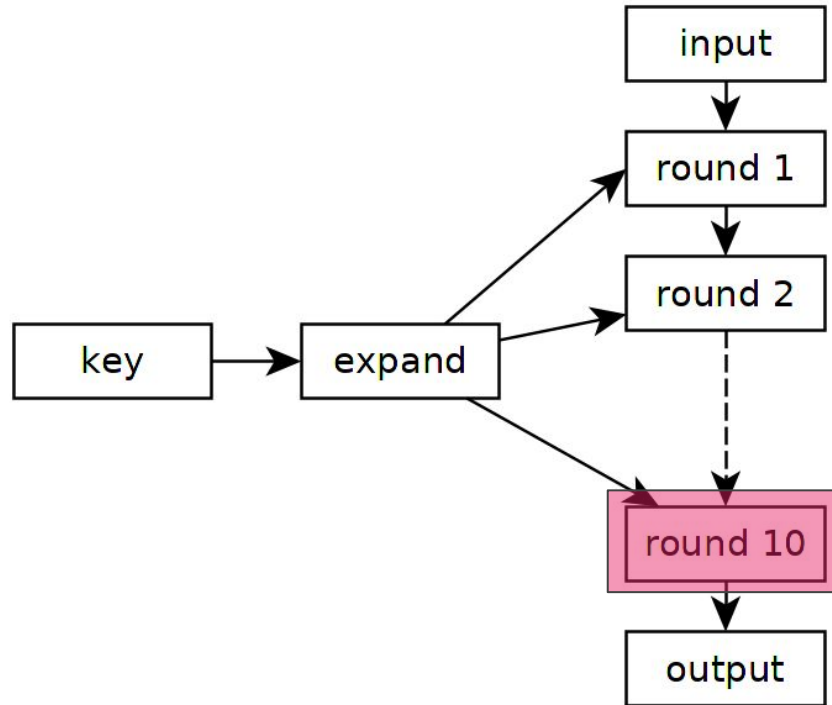
Advanced Encryption Standard

— — —

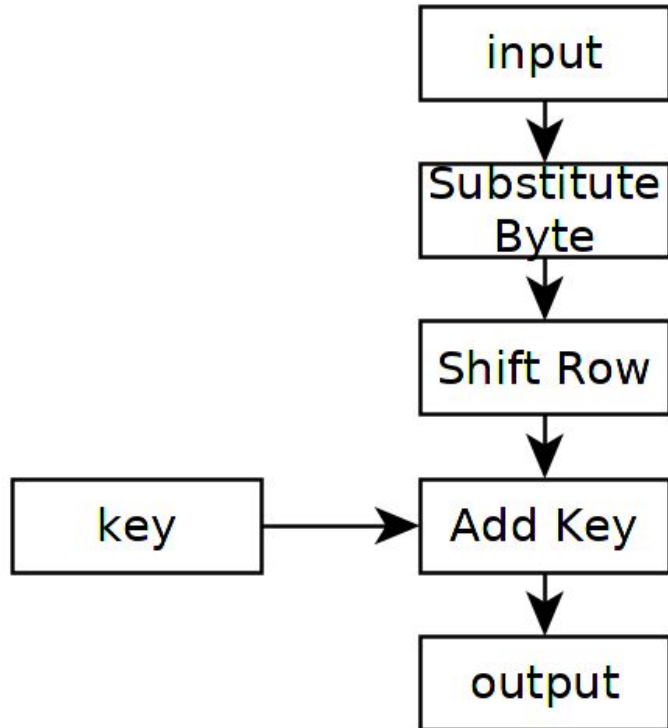


Advanced Encryption Standard

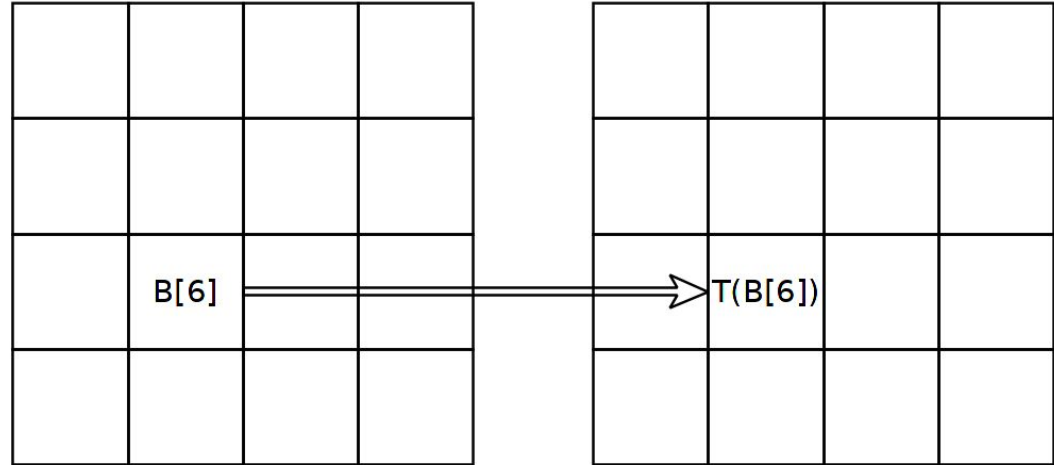
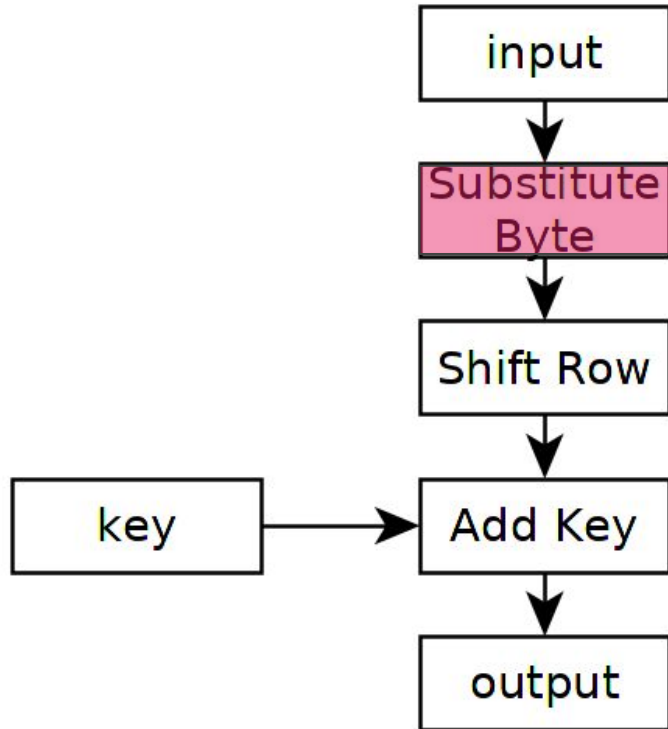
— — —



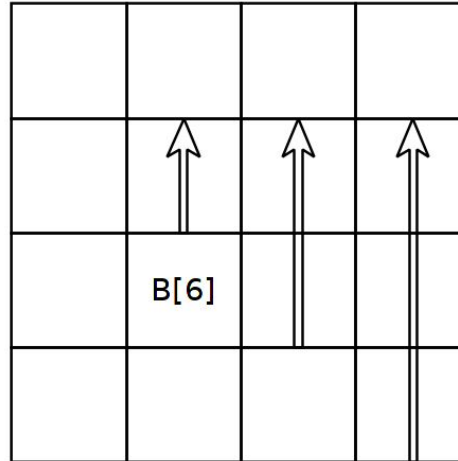
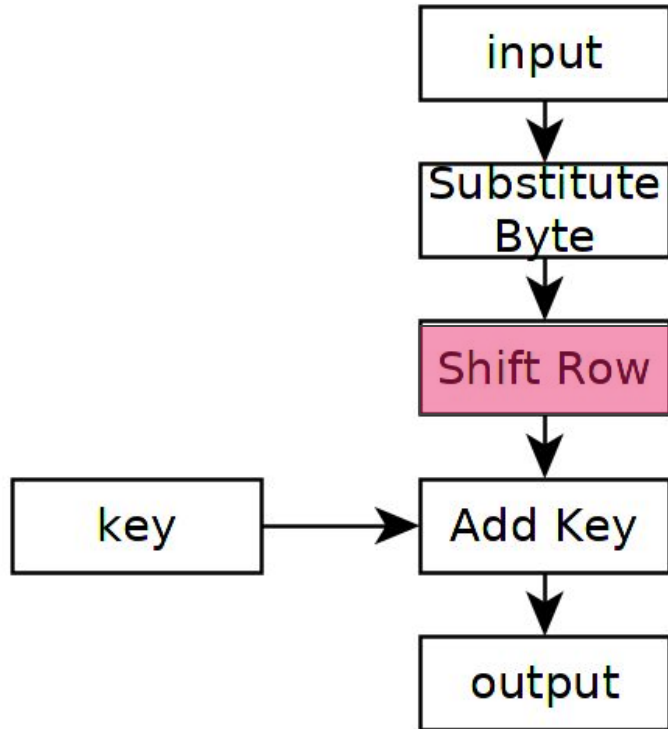
Advanced Encryption Standard: last round



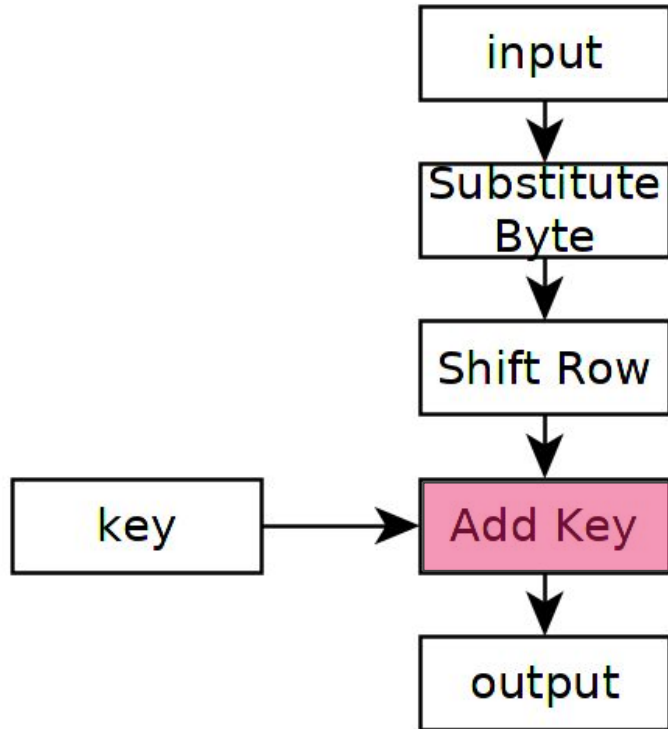
Advanced Encryption Standard: last round



Advanced Encryption Standard: last round



Advanced Encryption Standard: last round



	T(B[6]) xor K[5]		

AES Vulnerability

- $C[5] = T(B[6]) \oplus K[5]$
- Each byte of C depends on one byte of B and one byte of K
 - $T:[0 \dots 255] \rightarrow [0 \dots 255]$ implemented using a look-up table (Sbox)
 - 256 entries
- Let i be the index of the accessed entry ($i = B[6]$)
 - Focus on one byte: $C = T[i] \oplus K$
 - $K = C \oplus T[i]$
- Attacker model
 - Controls B (but does not observe it), observes C , knows T
 - shares cache
 - Goal: find i to discover C
- 1000 Encryptions $\Rightarrow$ Key

A small example

- Let $C[j] = M[P[j]] + K[j]$
- Plaintext, Key, and Ciphertext are 4 bytes (i.e. process repeated)
- M is a map of 256 integers (each integer is 4 bytes)
 - $256 \times 4 = 1024$ bytes
- Is this code non-interferent?

A small example

- Let $C[j] = M[P[j]] + K[j]$
- Plaintext, Key, and Ciphertext are 4 bytes (i.e. process repeated)
- M is a map of 256 integers (each integer is 4 bytes)
 - $256 \times 4 = 1024$ bytes
- Is this code non-interferent?
 - We declassify $M[P[j]] + K[j]$

A small example

- Let $C[j] = M[P[j]] + K[j]$
- Plaintext, Key, and Ciphertext are 4 bytes (i.e. process repeated)
- M is a map of 256 integers (each integer is 8 bytes)
 - $256 \times 8 = 2048$ bytes
- Cache lines are 64 bytes
 - $64/8$ elements for line = 8
 - $256/8 = 32$ lines

For simplicity suppose that $M[i]=i$

A small example

— — —

- Let $C = M[P] + K$
- M
 - 8 elements for line
 - 32 lines

For simplicity suppose that $M[i]=i$

$C = 0:1:0:16$

$\text{Lines} = \{0, 2, 7\}$

$C = 0:7:4:32$

$\text{Lines} = \{0, 2, 4, 9\}$

$C = 1:7:32:6$

$\text{Lines} = \{0, 4, 7, 9\}$

$C = 1:0:0:16$

$\text{Lines} = \{0, 2, 8\}$

A small example

— — —

- Let $C = M[P] + K$
- M
 - 8 elements for line
 - 32 lines

For simplicity suppose that $M[i]=i$

Lines for 0: {0,2}

0 =
{0,1,2,3,4,5,6,7,16,17,18,19,20,21,
22,23}+K

Lines for 1: {0}

1 = {0,1,2,3,4,5,6,7}+K

$C = 0:1:0:16$

Lines = {0, 2, 7}

$C = 0:7:4:32$

Lines = {0, 2, 4, 9}

$C = 1:7:32:6$

Lines = {0, 4, 7, 9}

$C = 1:0:0:16$

Lines = {0, 2, 8}

A small example

— — —

- Let $C = M[P] + K$
- M
 - 8 elements for line
 - 32 lines

For simplicity suppose that $M[i]=i$

$1-0 = \{0,1,2,3,4,5,6,7\}+K -$
 $0,1,2,3,4,5,6,7,16,17,18,19,20,21,2$
 $2,23\}-K$

$C = 0:1:0:16$

$\text{Lines} = \{0, 2, 7\}$

$C = 0:7:4:32$

$\text{Lines} = \{0, 2, 4, 9\}$

$C = 1:7:32:6$

$\text{Lines} = \{0, 4, 7, 9\}$

$C = 1:0:0:16$

$\text{Lines} = \{0, 2, 8\}$

A small example

— — —

- Let $C = M[P] + K$
- M
 - 8 elements for line
 - 32 lines

For simplicity suppose that $M[i]=i$

$1 = \{0,1,2,3,4,5,6,7\} -$
 $0,1,2,3,4,5,6,7,16,17,18,19,20,21,2$
 $2,23\}$

$C = 0:1:0:16$

$\text{Lines} = \{0, 2, 7\}$

$C = 0:7:4:32$

$\text{Lines} = \{0, 2, 4, 9\}$

$C = 1:7:32:6$

$\text{Lines} = \{0, 4, 7, 9\}$

$C = 1:0:0:16$

$\text{Lines} = \{0, 2, 8\}$

A small example

— — —

- Let $C = M[P] + K$
- M
 - 8 elements for line
 - 32 lines

For simplicity suppose that $M[i]=i$

$1 = \{0, \mathbf{1}, 2, 3, 4, 5, 6, 7\} -$
 $\mathbf{0}, 1, 2, 3, 4, 5, 6, 7, 16, 17, 18, 19, 20, 21, 2$
 $2, 23\}$

$C = 0:1:0:16$

$\text{Lines} = \{0, 2, 7\}$

$C = 0:7:4:32$

$\text{Lines} = \{0, 2, 4, 9\}$

$C = 1:7:32:6$

$\text{Lines} = \{0, 4, 7, 9\}$

$C = 1:0:0:16$

$\text{Lines} = \{0, 2, 8\}$

A small example

— — —

- Let $C = M[P] + K$
- M
 - 8 elements for line
 - 32 lines

For simplicity suppose that $M[i]=i$

$1 = \{0, 1, \mathbf{2}, 3, 4, 5, 6, 7\} -$
 $0, \mathbf{1}, 2, 3, 4, 5, 6, 7, 16, 17, 18, 19, 20, 21, 2$
 $2, 23\}$

$C = 0:1:0:16$

$\text{Lines} = \{0, 2, 7\}$

$C = 0:7:4:32$

$\text{Lines} = \{0, 2, 4, 9\}$

$C = 1:7:32:6$

$\text{Lines} = \{0, 4, 7, 9\}$

$C = 1:0:0:16$

$\text{Lines} = \{0, 2, 8\}$

A small example

— — —

- Let $C = M[P] + K$
- M
 - 8 elements for line
 - 32 lines

For simplicity suppose that $M[i]=i$

$1 = \{0,1,2,3,4,5,6,7\} -$
 $0,1,2,3,4,5,6,7,16,17,18,19,20,21,2$
 $2,23\}$

$C = 0:1:0:16$

$\text{Lines} = \{0, 2, 7\}$

$C = 0:7:4:32$

$\text{Lines} = \{0, 2, 4, 9\}$

$C = 1:7:32:6$

$\text{Lines} = \{0, 4, 7, 9\}$

$C = 1:0:0:16$

$\text{Lines} = \{0, 2, 8\}$

A small example

— — —

- Let $C = M[P] + K$
- M
 - 8 elements for line
 - 32 lines

For simplicity suppose that $M[i]=i$

Lines for 0: {0}

0 = {0,1,2,3,4,5,6}+K

Lines for 1: {0}

1 = {1,2,3,4,5,6,7}+K

$C = 0:1:0:16$

Lines = {0, 2, 7}

$C = 0:7:4:32$

Lines = {0, 2, 4, 9}

$C = 1:7:32:6$

Lines = {0, 4, 7, 9}

$C = 1:0:0:16$

Lines = {0, 2, 8}

Attacker goal: find i (accessed SBox entry) from C

- We assume
 - Attacker can read C and probe the cache once for encryption
 - Several encryptions with the same key
 - E.g. attacker is a malicious Android application that send a data via a VPN
 - Attacker primes/probes all D-cache lines
 - to identify cache lines evicted
- Problems
 - A cache line can be evicted due to other memory accesses (i.e. stack)
 - The same cache line could be accessed for several byte indexes
 - The same cache line could be accessed by several rounds
 - Once we find the cache line, we must find the precise SBox index

SBox in cache

— — —

- Assume SBox aligned to cache line 0
- Assume D-cache line 64 byte
- SBox is a table of 256 words => 1KB
 - 16 SBox entries per line, 16 lines
- Last round accesses SBox 16 times
 - 10.30 is the expected number of cache lines accessed

Non-elimination method

Let L be a log of encryptions

(c, e) in L = cyphertext, cache-line accesses at least once

$L(j, v) = \{(c, e) \text{ in } L \text{ such that } c[j]=v\}$

- Subset of L such that j -th byte of cyphertext is v
- $c[j] = K[j] \oplus T[i]$, T is a permutation, and K is fixed
- all elements of $L(j, v)$ accessed the same cache line

$E(j, v) = \text{Intersect}(\{e \mid (c, e) \text{ in } L(j, v)\})$

- is not empty and contains the line accessed for byte j and value v in the last round

Non-elimination method

(c, e) in L are the logs

$L(j, v) = \{(c, e) \text{ in } L \text{ such that } c[j]=v\}$

$E(j, v) = \text{Intersect}(\{e \mid (c, e) \text{ in } L(j, v)\})$

$T\{e\} = \{T[i] \text{ such that } i \text{ in } e\}$

- Values of SBox T allocated in lines e

Set initially $P(j, v) = T\{E(j, v)\}$

- Possible values of SBox accessed while producing cyphertext v for byte index j

Non-elimination method: fix byte index j

Take two different values v_1, v_2 such $L(v_1, j)$ and $L(v_2, j)$ are not empty

- $v_1 = K[j] \oplus T[i \text{ for } v_1]$
- $v_2 = K[j] \oplus T[i \text{ for } v_2]$

Therefore $v_1 \oplus v_2 = T[i \text{ for } v_1] \oplus T[i \text{ for } v_2]$

Shrink P as follows

- $P'(j, v_1) = \{b_1 \text{ in } P(k, v_1) \mid \text{exists } b_2 \text{ in } P(k, v_2) \text{ such that } v_1 \oplus v_2 = b_1 \oplus b_2\}$
- $P'(j, v_2) = \{b_2 \text{ in } P(k, v_2) \mid \text{exists } b_1 \text{ in } P(k, v_1) \text{ such that } v_1 \oplus v_2 = b_1 \oplus b_2\}$

Non-elimination method: fix byte index j

- When there is a value v such that $P(j,v)$ contains a single value
- Byte j of k has been recovered
- $v = K[j] \oplus T[i \text{ for } v]$
 - $K[j] = v \oplus T[i \text{ for } v]$
 - $K[j] = v \oplus P(j,v)[0]$
- Repeat attack for every byte index (no need of new logs)
- Usually 850 encryptions
 - Less than 1 second

Countermeasures

— — —

Eliminating the side channel

— — —

- Totally disable caches
- Disable caches for the victim
 - For trace driven and access driven attacks
- Disable caches for the attacker
 - To prevent him to prime/probe
- Huge performance impact

Cache flushing

- Only for access driven attacks
 - Flush at the beginning
 - Flush at exiting
- If attacker shares memory, then flush at beginning and exiting
 - Attacker could prime the cache and measure execution time of the victim
 - Attacker could measure if something is in the cache after the execution of the victim

Cache partitioning (colouring)

- For access driven and trace driven attacks
- Usually via MMU.
- Assume 8 KB cache, 4KB pages, 2 processes
 - Even pages are reserved for process 1
 - i.e. 100, 8K + 100, 2M + 16K + 100
 - Odd pages are reserved for process 2
 - I.e. 4K + 100, 10K + 100, 2M + 18K + 100
- Attacker cannot monitor cache accesses
- Problems
 - Under-utilization of caches
 - Difficult for several processes

Actively eliminating the side channel

- For access driven attacks
 - Victim can explicitly evicts all lines that can leak secrets (i.e. SBox) when exiting
 - Victim can normalize the cache by priming it with data that can be accessed
-
- For timing channels, victim can wait until its Worst Case Execution Time

Reducing side-channel bandwidth

— — —

- Introduce noise
 - Trace-driven and access driven: by accessing random data
 - Timing-channels: by adding random computations
- Reduce precision in measuring time
 - E.g. introduce random noise
 - (Attacker could access an external source for timing, like GPS signal or network protocols)

Randomization

— — —

- Randomize position of data structure
 - e.g. $\&M' = \&M + r$
- Randomize layout of data structure
 - e.g. $M[i] = M[\text{pi}(i)]$
- Randomize code
 - e.g. Change order of bytes in encryption

$$C[1] = M[P[1]] + K[1]$$

$$C[0] = M[P[0]] + K[0]$$

Detection

— — —

- Identify a process that primes caches:
 - At context switch cache contains suspicious (unrelated) addresses
- Identify a process that probes caches:
 - Frequent syscall requests to access system time

Constant time (power consumption, etc) programming

- Cache state depends on sequence of memory accesses (loads, stores, instruction fetches) performed
- Execution time depends on sequence of memory accesses and sequence of instruction executed
- Make these constants
- Constant time programming
 - Never branch on secret
 - Ensure that memory accesses are secret-independent
 - Usually too restrictive

Modular exponentiation (i.e. RSA)

```
function modular_pow(base, exponent, modulus)
  if modulus = 1 then return 0
  result := 1
  base := base mod modulus
  while exponent > 0
    if (exponent mod 2 == 1):
      result := (result * base) mod modulus

    V = (exponent mod 2) * base + 1 - (exponent mod 2)
    result := (result * V) mod modulus
    exponent := exponent >> 1
    base := (base * base) mod modulus
  return result
```

Other forms of side channels

Browser fingerprinting: what & why?

— — —

- Passive/active collection of browser + device characteristics to create a unique identifier (a “fingerprint”).
- User-Agent, screen size & color depth, timezone, installed fonts, list of plugins, audioContext features, platform, language.
- Enables cross-site tracking even when cookies are blocked or cleared
- In large studies (4M+ fingerprints), over 80% of collected browser fingerprints were unique in the dataset.
- Around 90% of attributes remain stable across months, meaning uniqueness persists across sessions.
- Recent studies (PETs Symposium 2025) show uniqueness can vary by age, income, and other demographics

How to reduce fingerprintability

— — —

- Choose privacy-focused browsers: Brave with anti-fingerprinting on.
- Block third-party scripts, disable unnecessary plugins, avoid granting wide JS access where possible
- Use extensions carefully
- Make your fingerprint common: Keep default settings, use common window sizes, don't install unique fonts or plugins
- Operational options: Use private VMs, containerised browsing, or always browse in a hardened profile for sensitive tasks.
- Legal/ethical note: For site owners, disclose tracking, get consent when required, and prefer privacy-preserving analytics.

Network analysis: Video identification

— — —

HTTP Adaptive Streaming (DASH):
Videos are split into segments of
variable size / bitrates.

The pattern of chunk sizes / timing
leaks info.

Bytes-Per-Second / Bits-Per-Peak
(BPS / BPP): Many works extract a
time series of bytes or peak
traffic bursts to fingerprint a
video.

CNNs or sequential models applied
to traffic feature sequences to
classify video titles.

Recent work shows that even in LTE
networks, with sniffing of radio
signals, adversaries can identify
which video is being watched with
high accuracy (up to ~ 0.985).

Network analysis: SSH — Password extraction

— — —

SSH sends small packets for interactive typing; measuring packet arrival times reveals the time between keystrokes. Statistical models can turn timings into likely key sequences and recover passwords.

Protocol/implementation details may leak exact password length.

server/client can buffer or coalesce keystroke packets

Traffic shaping

Questions?

— — —